

SpecDraft: Speculative Decoding with a Tiny Draft Model

A. Example, B. Sample (fictional)

FICTIONAL TEST DOCUMENT - generated for the workshop; not a real paper.

Abstract

We present SpecDraft, a speculative decoding method that pairs a 68M-parameter draft model with a 7B-parameter target model. The draft proposes four tokens at a time and the target verifies them in a single forward pass. On a code-completion workload SpecDraft gives a 2.1x speed-up with identical outputs to the target model.

1 Introduction

Large language models generate one token per forward pass, so latency grows with output length. Speculative decoding hides part of this cost: a small model guesses ahead and a large model checks the guesses in parallel. The speed-up depends on how often the guesses are accepted.

2 Method

The draft model shares the tokenizer of the target. At each step it proposes $k = 4$ tokens. The target scores all proposals at once and keeps the longest prefix it agrees with, then adds one token of its own. No retraining of the target model is needed.

3 Results

On 5,000 Python completion requests the draft tokens were accepted 71% of the time. Mean latency fell from 840 ms to 400 ms per request, a 2.1x speed-up. Outputs were token-for-token identical to the target model, because every token is verified.

4 Limitations

We tested one model pair and one workload. Acceptance drops to 38% on natural-language prompts, where the speed-up shrinks to 1.2x. Memory use grows because two models are loaded on the same GPU.

5 Conclusion

A tiny draft model can make a large code model about twice as fast without changing its answers, as long as the draft predicts the target well.